



Inverted Pendulum Swing-Up and Stabilization Control

ΑΓΓΕΛΟΣ ΚΑΜΑΡΙΑΔΗΣ

ΕΡΩΤΗΜΑ 1 - ΣΗΜΕΙΑ ΙΣΟΡΡΟΠΙΑΣ ΚΑΙ ΓΡΑΜΜΙΚΟΠΟΙΗΣΗ

Υπολογισμός σημείων ισορροπίας

Ένα αυτόνομο σύστημα χωρίς εξωτερική είσοδο ($u = 0$), βρίσκεται σε κατάσταση ισορροπίας, όταν οι ρυθμοί μεταβολής των καταστάσεων του είναι μηδέν, δηλαδή $\dot{w} = 0$. Βασιζόμενοι λοιπόν στο μοντέλο των εξισώσεων (8) της εκφώνησης της άσκησης, έχουμε το διάνυσμα κατάστασης:

$$w = [x, \theta, \dot{x}, \dot{\theta}]^T = [w_1, w_2, w_3, w_4]^T$$

Θέτοντας: $\dot{w}_1 = \dot{w}_2 = \dot{w}_3 = \dot{w}_4 = 0$, παίρνουμε:

Από $f_1(w, 0) = 0 \Rightarrow w_3 = 0 \Rightarrow$ η ταχύτητα του cart είναι 0

Από $f_2(w, 0) = 0 \Rightarrow w_4 = 0 \Rightarrow$ η γωνιακή ταχύτητα του εκκρεμούς είναι 0

Αντικαθιστώντας όπου $u = 0$ και όπου $w_4 = 0$ στην $f_3(w, 0) = 0$, ο αριθμητής μηδενίζεται:

$$-mg\sin(w_2)\cos(w_2) = 0 \Rightarrow \sin(2w_2) = 0 \Rightarrow 2(w_2) = k\pi \Rightarrow w_2 = k\pi/2, \text{ όπου } k \in \mathbb{Z}$$

Αντικαθιστώντας όπου $u = 0$ και όπου $w_4 = 0$ στην $f_4(w, 0) = 0$, ο αριθμητής

$$\text{μηδενίζεται: } -18(M+2m)g\sin(w_2) = 0 \Rightarrow \sin(w_2) = 0 \Rightarrow w_2 = k\pi$$

Για να ισχύουν και τα δύο ταυτόχρονα, πρέπει $w_2 = k\pi$. Το $x(w_1)$ μπορεί να είναι οποιοδήποτε εφόσον δεν εμφανίζεται στις εξισώσεις f_3 και f_4 . Άρα τα σημεία ισορροπίας είναι:

$$w_{eq} = [x_e, k\pi, 0, 0]^T$$

Έτσι, συμπεραίνουμε πως το σύστημα ισορροπεί με το cart ακίνητο σε οποιαδήποτε θέση x_e , ενώ παράλληλα το εκκρεμές δύναται να είναι είτε εντελώς όρθιο ($k \in \mathbb{Z}: k\%2 = 0 \Rightarrow$ ασταθές σημείο) είτε εντελώς προς τα κάτω ($k \in \mathbb{Z}: k\%2 \neq 0 \Rightarrow$ ευσταθές σημείο).

Γραμμικοποίηση

Η γραμμικοποίηση γύρω από το $w = [0, 0, 0, 0]^T$, απαιτεί τον υπολογισμό των Ιακωβιανών πινάκων: A (πίνακας καταστάσεων) και B (πίνακας εισόδων).

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{\partial f_3}{\partial w_2} & 0 & 0 \\ 0 & \frac{\partial f_4}{\partial w_2} & 0 & 0 \end{bmatrix} \text{ και } B = \begin{bmatrix} 0 \\ 0 \\ \frac{\partial f_3}{\partial u} \\ \frac{\partial f_4}{\partial u} \end{bmatrix}$$

Εφόσον το σημείο ισορροπίας μας είναι το $w = [0,0,0,0]^T$, βρισκόμαστε στην περιοχή όπου η γωνία $\theta \approx 0$ και η ταχύτητα $\dot{\theta} \approx 0$. Βάσει του αναπτύγματος Taylor, κρατάμε μόνο τους πρωτοβάθμιους όρους, άρα: $\sin\theta \approx \theta$, $\cos\theta \approx 1$ και $\dot{\theta}^2 \approx 0$. Αντικαθιστώντας αυτά στις δύο μη γραμμικές εξισώσεις κίνησης, παίρνουμε το γραμμικό σύστημα:

$$(M + 2m)\ddot{x} + \frac{3}{2}ml\ddot{\theta}\cos\theta - \frac{3}{2}ml\dot{\theta}^2\sin\theta = F, F = u$$

$$\Rightarrow (M + 2m)\ddot{x} + \frac{3}{2}ml\ddot{\theta} = u$$

$$9\ddot{x}\cos\theta + 8l\ddot{\theta} - 9g\sin\theta = 0$$

$$\Rightarrow \ddot{x} = g\theta - \frac{8}{9}l\ddot{\theta}$$

Αντικαθιστώντας το $\ddot{x}$ στην πρώτη εξίσωση παίρνουμε:

$$(M + 2m)(g\theta - \frac{8}{9}l\ddot{\theta}) + \frac{3}{2}ml\ddot{\theta} = u$$

$$\Rightarrow \ddot{\theta} = \frac{18(M + 2m)g}{l(16M + 5m)}\theta - \frac{18}{l(16M + 5m)}u$$

Τώρα, λύνουμε την δεύτερη εξίσωση ως προς $\ddot{\theta}$ και παίρνουμε:

$$\ddot{\theta} = \frac{9g\theta - 9\ddot{x}}{8l}$$

Το αντικαθιστούμε στην πρώτη εξίσωση και εν τέλη παίρνουμε:

$$\ddot{x} = -\frac{27mg}{16M + 5m}\theta + \frac{16}{16M + 5m}u$$

Τέλος, για να κατασκευάσουμε τους πίνακες A και B, πρέπει να υπολογίσουμε τα: $\frac{\partial f_3}{\partial w_2}$ και $\frac{\partial f_4}{\partial w_2}$.

Γνωρίζουμε ότι το διάνυσμα κατάστασης είναι το $w = [x, \theta, \dot{x}, \dot{\theta}]^T = [w_1, w_2, w_3, w_4]^T$. Το σύστημα έχει τη μορφή $\dot{w} = Aw + Bu$. Άρα, με βάση όσα βρήκαμε, οι πίνακες είναι οι:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{27mg}{16M+5m} & 0 & 0 \\ 0 & \frac{18(M+2m)g}{l(16M+5m)} & 0 & 0 \end{bmatrix} \text{ και } B = \begin{bmatrix} 0 \\ 0 \\ \frac{16}{16M+5m} \\ -\frac{18}{l(16M+5m)} \end{bmatrix}$$

ΕΡΩΤΗΜΑ 2 - LQR CONTROLLER

Ο LQR (Linear Quadratic Regulator) controller, είναι ένας ελεγκτής βέλτιστου ελέγχου, ο οποίος ψάχνει να βρει έναν νόμο ελέγχου της μορφής $u = -Kw$, που θα ελαχιστοποιεί την συνάρτηση:

$$J = \int_0^{\infty} (w^T Q w + u^T R u) dt$$

Για να βρούμε τον πίνακα κερδών K , χρησιμοποιούμε την βιβλιοθήκη της python `scipy.linalg`. Ο παρακάτω κώδικας, λύνει την εξίσωση riccati:

$$A^T P + P A - P B R^{-1} B^T P + Q = 0$$

με σκοπό να υπολογίσει το P από το οποίο τελικά υπολογίζουμε τον πίνακα κερδών ως εξής:

$$K = R^{-1} B^T P$$

```
1 import numpy as np
2 import scipy.linalg as la
3
4 M = 1.0
5 m = 0.1
6 l = 0.5
7 g = 9.81
8 den = 16 * M + 5 * m
9 A = np.array([
10     [0, 0, 1, 0],
11     [0, 0, 0, 1],
12     [0, -27 * m * g / den, 0, 0],
13     [0, 18 * (M + 2 * m) * g / (l * den), 0, 0]
14 ])
15 B = np.array([
16     [0],
17     [0],
18     [16 / den],
19     [-18 / (l * den)]
20 ])
21 Q = np.diag([10, 1000, 1, 10])
22 R = np.array([[0.1]])
23 P = la.solve_continuous_are(A, B, Q, R)
24 K = la.inv(R) @ B.T @ P
25 print("Ο πίνακας κέρδους LQR (K) είναι:")
26 print(np.round(K, decimals=4))
```

Το αποτέλεσμα του παραπάνω κώδικα, δηλαδή ο πίνακας κερδών, φαίνεται παρακάτω.

```
Run main x
/Users/angeloskamaradias/Library/Mobile Documents/com-apple-CloudDocs/ΣΗΜΜΥ/3ο Έτος/6ο Εξάμηνο/Σχεδίαση Συστημάτων Αυτομάτου Έλεγχου/Projects/Project 4/code/.venv/bin/python /
0 πίνακας κέρδους LQR (K) είναι:
[[ -10.    -138.6017  -16.3882  -22.2821]]
Process finished with exit code 0
```

Άρα, ο πίνακας κερδών είναι ο $K = [-10 \ -138,6016 \ -16,3882 \ -22,2821]$.

Η συμπεριφορά του συστήματος κλειστού βρόχου, καθορίζεται από την ισορροπία μεταξύ της ακρίβειας ιχνηλάτησης και της απαιτούμενης ενέργειας ελέγχου, η οποία ελέγχεται πλήρως από τους πίνακες βαρών Q και R . Ειδικότερα, ο πίνακας Q , τιμωρεί τις αποκλίσεις των μεταβλητών κατάστασης, δηλαδή τη θέση του cart, τη γωνία του εκκρεμούς και οι ταχύτητές τους. Επομένως η αύξηση των τιμών του αναγκάζει τον ελεγκτή να επιδιώξει ταχύτερο μηδενισμό των αντίστοιχων σφαλμάτων, καθιστώντας το σύστημα πιο εστιασμένο στην ισορροπία. Αντίθετα, ο πίνακας R τιμωρεί το κόστος της προσπάθειας ελέγχου, δηλαδή την ασκούμενη δύναμη. Ένα μικρό R δημιουργεί έναν επιθετικό ελεγκτή που καταναλώνει μεγάλη ενέργεια για να διορθώσει ακαριαία κάθε διαταραχή, ενώ ένα μεγάλο R οδηγεί σε έναν συντηρητικό ελεγκτή με ομαλή και πιο αργή απόκριση, προστατεύοντας μεν τους φυσικούς ενεργοποιητές από κορεσμό, αλλά ρισκάροντας δε την πτώση του εκκρεμούς. Αυτές οι επιλογές σχεδιασμού, αντικατοπτρίζονται άμεσα στο τελικό βέλτιστο κέρδος K . Η αύξηση της επιρροής του Q έναντι του R , δηλαδή ο μεγάλος λόγος Q/R , μεγιστοποιεί την απόλυτη τιμή των στοιχείων του πίνακα K , μετατοπίζοντας τους πόλους του κλειστού βρόχου πιο αριστερά στο αριστερό μιγαδικό ημιεπίπεδο για ταχύτερη απόσβεση, ενώ η αύξηση του R συρρικνώνει τις τιμές του K , αφήνοντας τους πόλους πιο κοντά στον φανταστικό άξονα, πράγμα που σημαίνει ασθενέστερη δράση ελέγχου και μεγαλύτερο χρόνο ηρεμίας του συστήματος.

ΕΡΩΤΗΜΑ 3 - ΣΧΕΔΙΑΣΗ ΘΕΣΗ CART ΚΑΙ ΓΩΝΙΑΣ ΕΚΚΡΕΜΟΥΣ ΣΥΝΑΡΤΗΣΕΙ ΤΟΥ ΧΡΟΝΟΥ

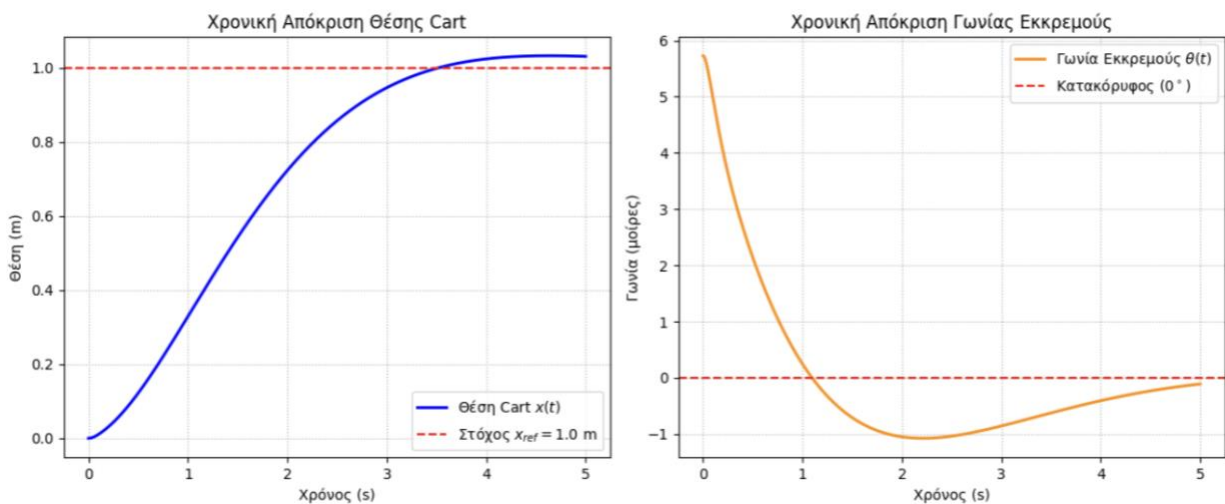
Παρακάτω, απεικονίζεται ο κώδικας με τον οποίο σχεδιάστηκαν τόσο η θέση του cart, όσο και η γωνιά του εκκρεμούς, συναρτήσει του χρόνου.

```
1 import numpy as np
2 import scipy.linalg as la
3 import matplotlib.pyplot as plt
4 from scipy.integrate import solve_ivp
5
6 M = 1.0
7 m = 0.1
8 l = 0.5
9 g = 9.81
10 den = 16 * M + 5 * m
11 A = np.array([
12     [0, 0, 1, 0],
13     [0, 0, 0, 1],
14     [0, -27 * m * g / den, 0, 0],
15     [0, 18 * (M + 2 * m) * g / (l * den), 0, 0]
16 ])
17 B = np.array([
18     [0],
19     [0],
20     [16 / den],
21     [-18 / (l * den)]
22 ])
23 Q = np.diag([10, 1000, 1, 10])
24 R = np.array([[0.1]])
25 P = la.solve_continuous_are(A, B, Q, R)
26 K = la.inv(R) @ B.T @ P
27 x_ref = 1.0
28 w_ref = np.array([x_ref, 0.0, 0.0, 0.0])
29 def closed_loop_dynamics(t, w): 1 usage
30     u = -K @ (w - w_ref)
31     dw = A @ w + B @ u
32     return dw
33 w0 = np.array([0.0, 0.1, 0.0, 0.0])
34 t_span = (0, 5)
35 t_eval = np.linspace(t_span[0], t_span[1], num=1000)
36 sol = solve_ivp(closed_loop_dynamics, t_span, w0, t_eval=t_eval)
```

```

37 plt.figure(figsize=(12, 5))
38 plt.subplot(*args: 1, 2, 1)
39 plt.plot(*args: sol.t, sol.y[0], label='Θέση Cart  $x(t)$ ', color='blue', linewidth=2)
40 plt.axhline(x_ref, color='red', linestyle='--', label=f'Στόχος  $x_{\{ref\}} = \{x_{ref}\}$  m')
41 plt.title('Χρονική Απόκριση Θέσης Cart')
42 plt.xlabel('Χρόνος (s)')
43 plt.ylabel('Θέση (m)')
44 plt.grid(visible=True, linestyle=':')
45 plt.legend()
46 plt.subplot(*args: 1, 2, 2)
47 theta_deg = np.degrees(sol.y[1])
48 plt.plot(*args: sol.t, theta_deg, label='Γωνία Εκκρεμούς  $\theta(t)$ ', color='darkorange', linewidth=2)
49 plt.axhline(y=0, color='red', linestyle='--', label='Κατακόρυφος ( $\theta^{\circ}$ )')
50 plt.title('Χρονική Απόκριση Γωνίας Εκκρεμούς')
51 plt.xlabel('Χρόνος (s)')
52 plt.ylabel('Γωνία (μοίρες)')
53 plt.grid(visible=True, linestyle=':')
54 plt.legend()
55 plt.tight_layout()
56 plt.show()

```



Από τα πιο πάνω γραφήματα, συμπεραίνουμε τα ακόλουθα. Από τη χρονική απόκριση του συστήματος για μια βαθιά επιθυμητή θέση $x_{ref} = 1$, παρατηρούμε αρχικά την δυναμική συμπεριφορά non - minimum phase, όπου το καροτσάκι ενδέχεται να κινηθεί στιγμιαία προς την αντίθετη κατεύθυνση προκειμένου να δώσει στο εκκρεμές τη σωστή κλίση προς την πλευρά της κίνησης πριν το σταθεροποιήσει στον στόχο. Όσον αφορά τη φύση του γραμμικοποιημένου συστήματος, συμπεραίνουμε ότι αποτελεί μια εξαιρετική αλλά αυστηρά τοπική προσέγγιση της πραγματικότητας. Εφόσον οι μετατοπίσεις είναι μικρές, η γωνία θ του εκκρεμούς διατηρείται κοντά στο μηδέν, επιβεβαιώνοντας πλήρως τις μαθηματικές υποθέσεις της γραμμικοποίησης και επιτρέποντας στον ελεγκτή LQR να λειτουργήσει άψογα. Αν ωστόσο απαιτηθεί μια μεγάλη και απότομη, το θεωρητικό γραμμικό μοντέλο θα υποδείξει επιτυχία επιτρέποντας ανεδαφικά μεγάλες γωνίες απόκλισης. Σε αυτές τις μεγάλες γωνίες, οι γραμμικές προσεγγίσεις καταρρέουν πλήρως, με αποτέλεσμα ο ελεγκτής να αποτυγχάνει και το πραγματικό, μη γραμμικό εκκρεμές να πέφτει. Συνεπώς, το γραμμικοποιημένο μοντέλο μάς παρέχει αξιόπιστο έλεγχο αποκλειστικά και μόνο σε μια περιορισμένη περιοχή λειτουργίας γύρω από το ασταθές σημείο ισορροπίας.

ΕΡΩΤΗΜΑ 4 - ΠΑΡΑΤΗΡΗΤΗΣ LUENBERGER

Στην πράξη, είναι σπάνιο και ακριβό να χρησιμοποιούμε αισθητήρες που μετρούν απευθείας την ταχύτητα. Για αυτό, συνήθως χρησιμοποιούμε αισθητήρες που μετρούν μόνο τη θέση του cart και τη γωνία του εκκρεμούς. Επομένως, χρειαζόμαστε έναν αλγόριθμο που να εκτιμά με μαθηματική ακρίβεια τις ταχύτητες, έχοντας τη δυνατότητα να δει μόνο πως αλλάζουν οι θέσεις στον χρόνο.

Εφόσον λοιπόν μετρούμε μόνο το w_1 (θέση) και το w_2 (γωνία), το διάνυσμα εξόδου του συστήματός μας είναι το y , όπου $y = Cw$. Ο πίνακας C είναι:

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Ο παρατηρητής Luenberger, διατηρεί ένα αντίγραφο του συστήματος και το διορθώνει συνεχώς, συγκρίνοντας την πραγματική μέτρηση με την εκτιμώμενη μέτρηση. Η δυναμική εξίσωση του παρατηρητή είναι:

$$\dot{\hat{w}} = A\hat{w} + Bu + L(y - C\hat{w})$$

Σκοπός μας είναι να βρούμε τον πίνακα κερδών L , έτσι ώστε το σφάλμα εκτίμησης $e = w - \hat{w}$ να πηγαίνει στο μηδέν. Η συμπεριφορά αυτού του σφάλματος καθορίζεται από τις ιδιοτιμές του πίνακα $A - LC$.

Η επιλογή των ιδιοτιμών του παρατηρητή και η διαχείριση του προβλήματος του θορύβου απαιτούν τον εντοπισμό μιας προσεκτικής ισορροπίας. Ως βασικός κανόνας σχεδίασης, για να προλάβει ο ελεγκτής LQR να αντλήσει εγκαίρως σωστές πληροφορίες, ο παρατηρητής πρέπει να συγκλίνει ταχύτερα από το ίδιο το σύστημα, γεγονός που υπαγορεύει την επιλογή πόλων 3 έως 5 φορές πιο γρήγορων (πιο αρνητικών) από τους πόλους κλειστού βρόχου του ελεγκτή ($A - BK$). Ωστόσο, όσο πιο γρήγοροι είναι οι πόλοι, τόσο μεγαλύτερα γίνονται τα στοιχεία του πίνακα L , με αποτέλεσμα, αν ο αισθητήρας εισάγει θόρυβο ($y = Cw + \text{noise}$), το τεράστιο μέγεθος του L να ενισχύει αυτόν τον θόρυβο σε ακραίο βαθμό (φαινόμενο noise amplification). Σε μια τέτοια περίπτωση, ο παρατηρητής παρερμηνεύει τον θόρυβο ως πραγματική κίνηση και εκτιμά λανθασμένες ταχύτητες, κάνοντας το σήμα ελέγχου u να ταλαντώνεται με επιθετικό τρόπο (φαινόμενο chattering), προκαλώντας εν τέλει την πτώση του εκκρεμούς. Συμπερασματικά, όπως τονίζει και η εκφώνηση της άσκησης, η σχεδίαση απαιτεί μια χρυσή τομή όπου οι πόλοι θα είναι αρκετά γρήγοροι για να παρακολουθούν άμεσα τη δυναμική του συστήματος, αλλά και επαρκώς αργοί ώστε να φιλτράρουν τις υψηλές συχνότητες του θορύβου των μετρήσεων.

Ο παρακάτω κώδικας, υπολογίζει το L εισάγει θόρυβο στον αισθητήρα γωνίας, χρησιμοποιεί την εκτιμώμενη κατάσταση για τον έλεγχο και σχεδιάζει την πραγματική, ενάντια στην εκτιμώμενη γωνιακή ταχύτητα για να παρατηρήσουμε πόσο καλά δουλεύει.

```

import numpy as np
import scipy.linalg as la
import scipy.signal as signal
import matplotlib.pyplot as plt

M = 1.0;
m = 0.1;
l = 0.5;
g = 9.81
den = 16 * M + 5 * m
A = np.array([
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, -27 * m * g / den, 0, 0],
    [0, 18 * (M + 2 * m) * g / (l * den), 0, 0]
])
B = np.array([[0], [0], [16 / den], [-18 / (l * den)]])
C = np.array([[1, 0, 0, 0], [0, 1, 0, 0]]) # Μετράμε x και θ
Q = np.diag([10, 1000, 1, 10])
R = np.array([[0.1]])
P = la.solve_continuous_are(A, B, Q, R)
K = la.inv(R) @ B.T @ P
cl_poles = la.eigvals(A - B @ K)
obs_poles = np.array([-15, -16, -17, -18])
res = signal.place_poles(A.T, C.T, obs_poles)
L = res.gain_matrix.T

print("Observer Gain L =\n", np.round(L, decimals= 2))

dt = 0.005
time = np.arange(start_or_stop: 0, stop: 3, dt)
n_steps = len(time)
w = np.zeros((4, n_steps))
w_hat = np.zeros((4, n_steps))
w[:, 0] = [0.0, 0.1, 0.0, 0.0]
w_hat[:, 0] = [0.0, 0.0, 0.0, 0.0]

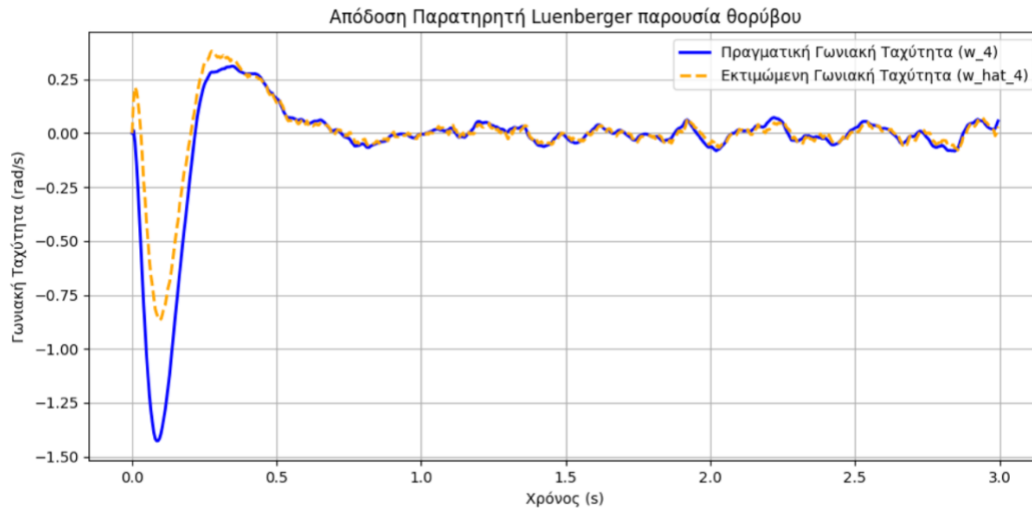
```

```

for i in range(1, n_steps):
    noise = np.random.normal(loc: 0, scale: 0.005, size: 2) # Τυπική απόκλιση 0.005
    y_noisy = C @ w[:, i - 1] + noise
    u = -K @ w_hat[:, i - 1]
    dw = A @ w[:, i - 1] + B @ u
    w[:, i] = w[:, i - 1] + dw * dt
    dw_hat = A @ w_hat[:, i - 1] + B @ u + L @ (y_noisy - C @ w_hat[:, i - 1])
    w_hat[:, i] = w_hat[:, i - 1] + dw_hat * dt

plt.figure(figsize=(10, 5))
plt.plot(*args: time, w[3, :], label='Πραγματική Γωνιακή Ταχύτητα (w_4)', color='blue', linewidth=2)
plt.plot(*args: time, w_hat[3, :], label='Εκτιμώμενη Γωνιακή Ταχύτητα (w_hat_4)', color='orange', linestyle='--', linewidth=2)
plt.title('Απόδοση Παρατηρητή Luenberger παρουσία θορύβου')
plt.xlabel('Χρόνος (s)')
plt.ylabel('Γωνιακή Ταχύτητα (rad/s)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Από το γράφημα της χρονικής απόκρισης, συμπεραίνουμε ότι ο παρατηρητής Luenberger λειτουργεί όπως πρέπει, καταφέροντας να εκτιμήσει με επιτυχία τη μη μετρήσιμη γωνιακή ταχύτητα του εκκρεμούς σε ρεαλιστικές συνθήκες θορύβου. Παρότι η διαδικασία εκκινεί με ένα εμφανές σφάλμα εκτίμησης λόγω άγνοιας των πραγματικών αρχικών συνθηκών, η εκτιμώμενη γωνιακή ταχύτητα κλειδώνει ταχύτατα πάνω στην πραγματική γωνιακή ταχύτητα μέσα στο πρώτο μισό δευτερόλεπτο, γεγονός που επιβεβαιώνει ότι οι πόλοι επιλέχθηκαν σωστά ώστε να είναι επαρκώς πιο γρήγοροι από το ίδιο το σύστημα. Στη συνέχεια, καθ' όλη τη διάρκεια της προσομοίωσης, διακρίνουμε στην εκτιμώμενη ταχύτητα μικρές και συνεχείς ταλαντώσεις οι οποίες αποτελούν την αναμενόμενη παρενέργεια της ενίσχυσης του θορύβου των αισθητήρων από τον πίνακα κερδών L . Το γεγονός ωστόσο, ότι αυτός ο θόρυβος διατηρείται σε χαμηλό αλλά ελεγχόμενο πλάτος και δεν αποσταθεροποιεί το σύστημα, αποδεικνύει ότι επιτεύχθηκε η ιδανική κατασκευαστική ισορροπία. Δηλαδή ο παρατηρητής είναι αρκετά γρήγορος για να συγκλίνει άμεσα, αλλά ταυτόχρονα αρκετά συντηρητικός ώστε να φιλτράρει τις υψηλές συχνότητες, αποτρέποντας τη μεταφορά καταστροφικού θορύβου στο σήμα ελέγχου του LQR.